

MÔN HỌC: Embedded Linux Systems

BÀI LAB: LAB-01 - Cấu hình Kernel & Khởi động Hệ thống Linux

Nhúng (ARMv7)

Nhóm: 3

Lớp: IC2009

Ngày thực hiện: 09/09/2026

Mục 1: THÔNG TIN THÀNH VIÊN NHÓM VÀ MÔI TRƯỜNG

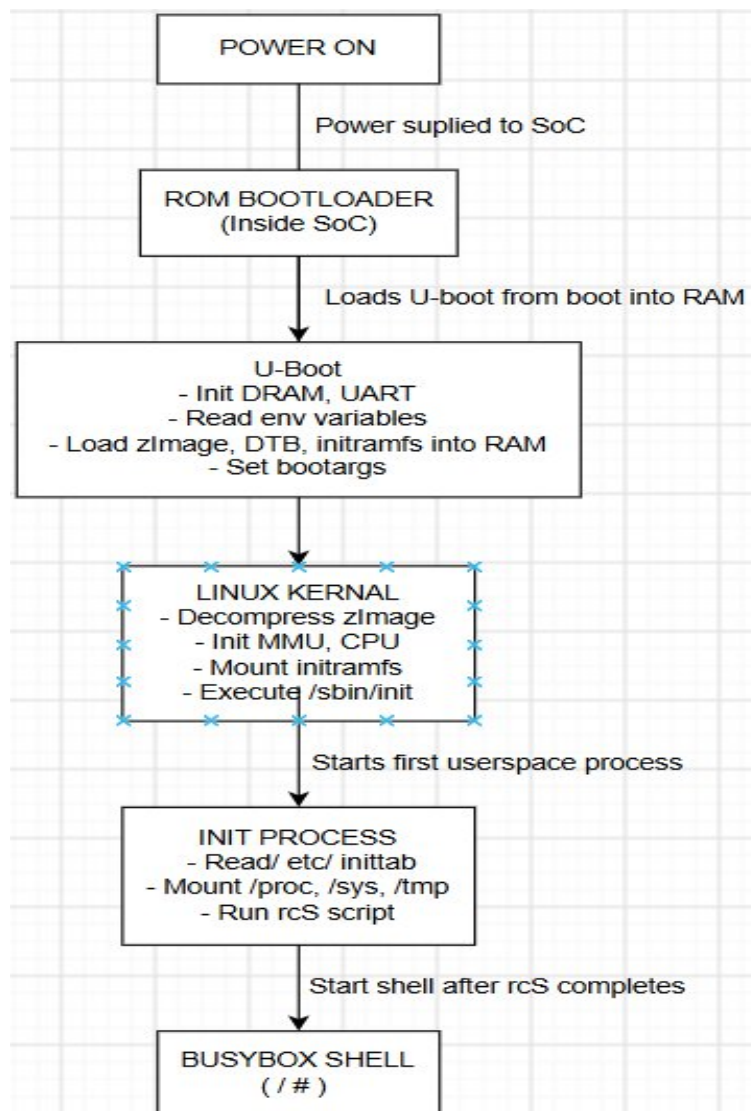
1.1. Group Members

STT	Họ và Tên	MSSV	Vai trò / Nhiệm vụ chính
1	Trần Gia Thành	SE203763	Nhóm trưởng / Cấu hình & Build Kernel (Phần A)
2	Châu Nhật Nguyên	SE203853	Build U-Boot & Thiết lập Bootargs (Phần B)
3	Võ Hoàng Trí	SE204383	Tạo BusyBox & Rootfs/initramfs (Phần C)
4	Nguyễn Đăng Khoa	SE204014	Kiểm thử Boot QEMU, Thu thập Log & Viết báo cáo

1.2. Môi trường thực hiện

- Hệ điều hành Host: Ubuntu 22.04 LTS (64-bit)
- RAM: 8 GB (Khuyến nghị)
- Dung lượng đĩa: 30 GB trống (Build kernel cần ~15 GB)
- QEMU: qemu-system-arm >= 4.2 (machine: vexpress-a9)
- ARM Toolchain: gcc-arm-linux-gnueabi >= 9.0 (ARMv7-A hard-float)
- Linux Kernel Source: v5.15 LTS
- U-Boot Source: v2022.04
- BusyBox: v1.35.0

Mục 2: SƠ ĐỒ TOÀN BỘ CHUỖI KHỞI ĐỘNG BOOT SEQUENCE DIAGRAM VÀ CÁC THAO TÁC



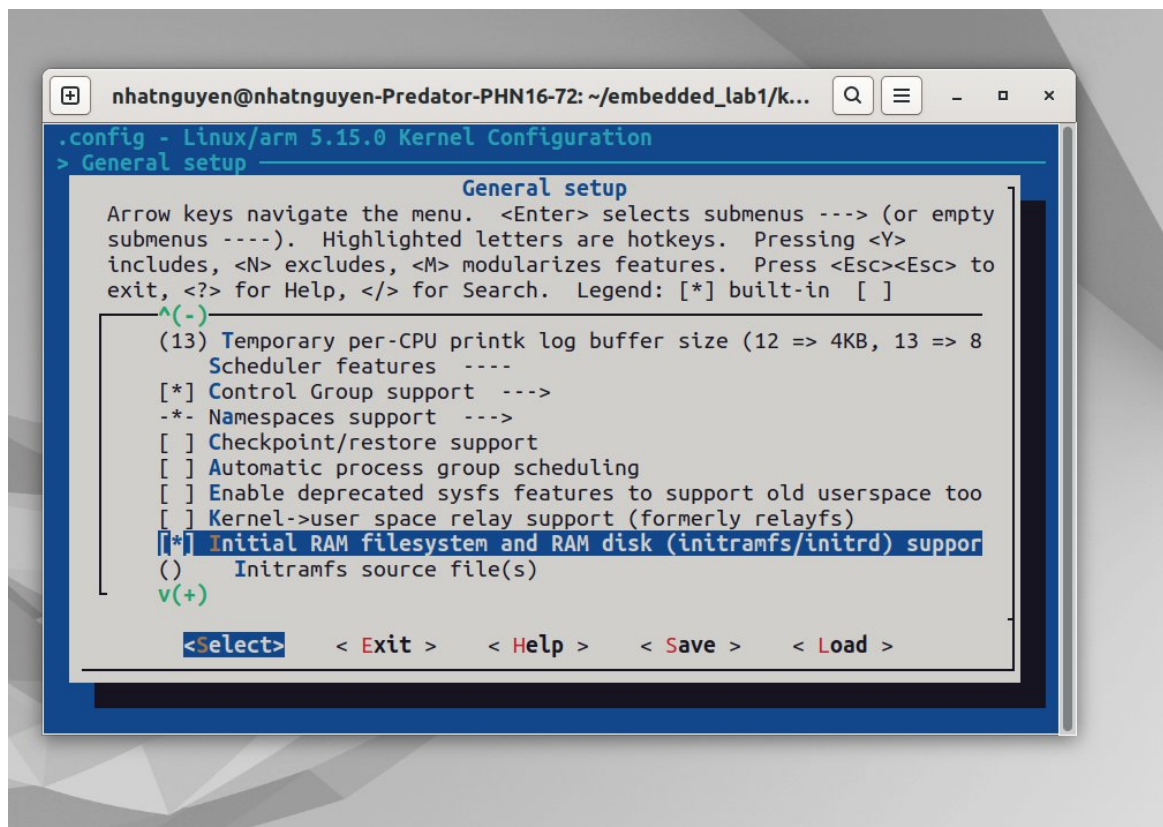
Mô tả chuỗi khởi động:

1. Power ON (Cấp nguồn): Hệ thống được cấp nguồn. CPU bắt đầu thực thi mã từ ROM bootloader (bên trong SoC).
2. ROM Bootloader: Khởi tạo phần cứng cơ bản và nạp bootloader chính (U-Boot) từ nguồn khởi động (thẻ SD, eMMC, hoặc mạng) vào RAM.
3. U-Boot: Khởi tạo DRAM, console serial, và các thiết bị ngoại vi. Nó đọc các biến môi trường (bootargs, bootcmd) và nạp kernel image (zImage), Device Tree Blob (vexpress-v2p-ca9.dtb), và initramfs vào RAM. Sau đó nhảy đến điểm entry của kernel.

4. Linux Kernel: Tự giải nén, khởi tạo bộ quản lý bộ nhớ (MMU), thiết lập CPU, và khởi tạo các subsystem (ngắt, timer, driver). Sau đó mount root filesystem từ initramfs và thực thi tiến trình init.
5. Tiến trình init (/sbin/init): Đọc /etc/inittab, mount /proc, /sys, và /tmp, và thực thi script rcS.
6. BusyBox Shell: Sau khi rcS hoàn tất, BusyBox hiển thị dấu nhắc shell (/ #) cho người dùng tương tác.

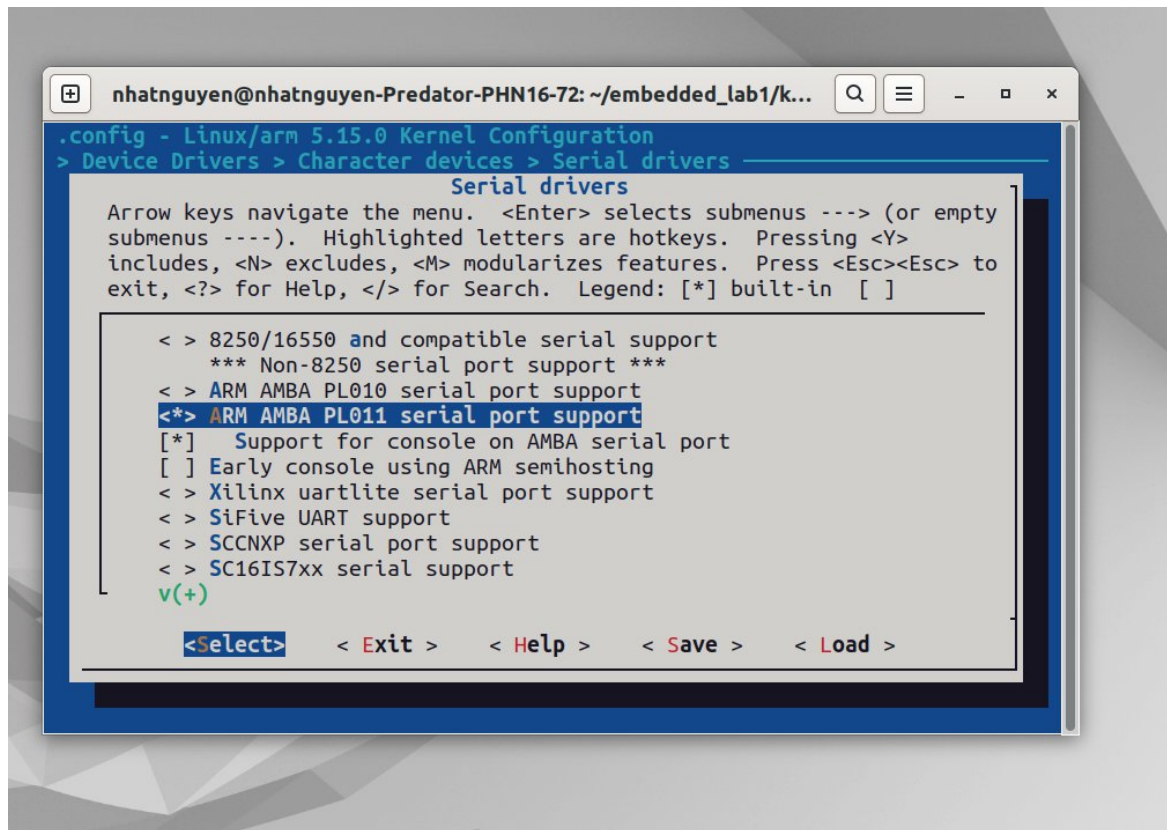
Mục 3:

Phần A: Output của lệnh make versatile_defconfig, log cuối build kernel, kết quả ls -lh zImage



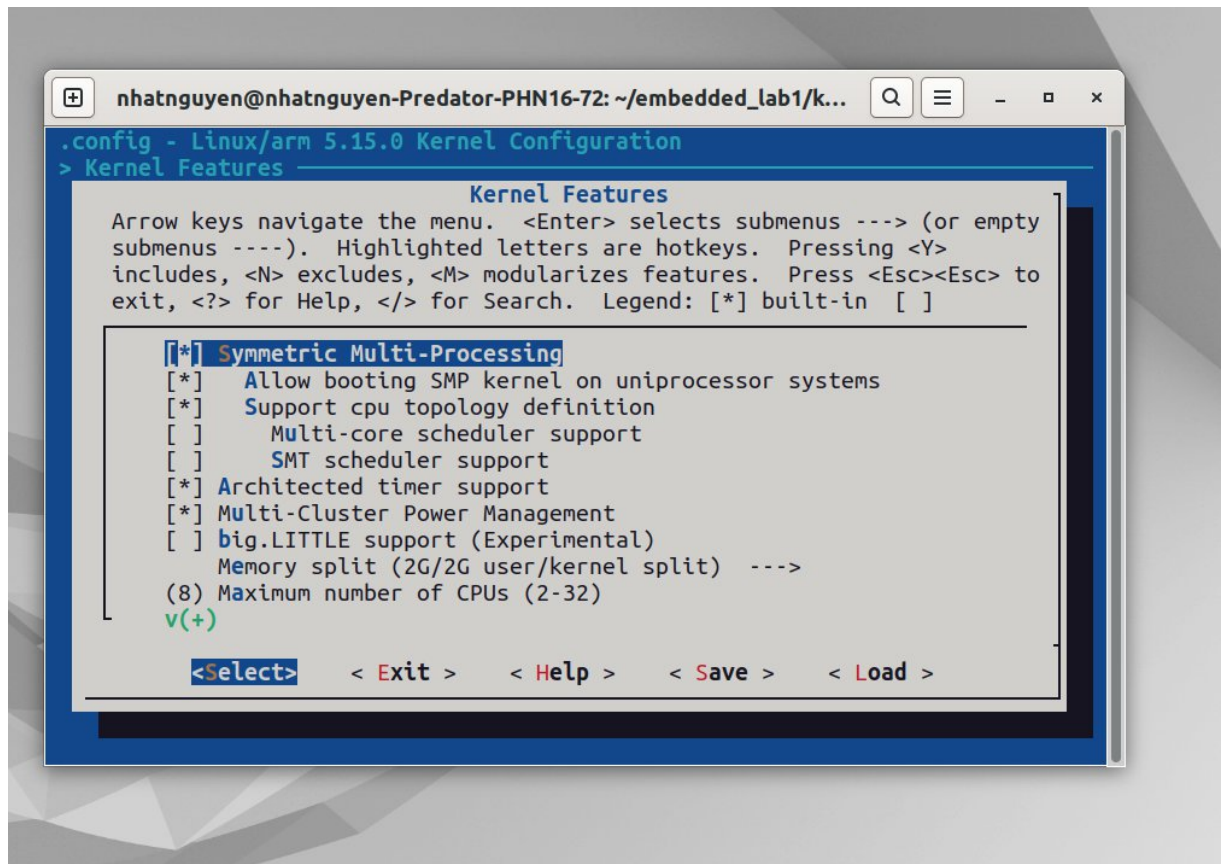
Bật hỗ trợ initramfs

- Bật tùy chọn này (đánh dấu [*])
- Vào: General setup → Initial RAM filesystem and RAM disk (initramfs/initrd) support



Cấu hình console output cho vexpress-a9

- Vào: Device Drivers → Character devices → Serial drivers
- Đảm bảo: ARM AMBA PL011 serial port support được bật [*]



Bật SMP cho Cortex-A9

- Vào: Kernel Features → Symmetric Multi-Processing
- Bật [*] SMP (Symmetric Multi-Processing) (vexpress-a9 trong QEMU hỗ trợ tối đa 4 core)

Log cuối build kernel, kết quả ls -lh zImage

```
nhatnguyen@nhatnguyen-Predator-PHN16-72: ~/embedded_lab1/kernel/linux-5.15
LD      .tmp_vmlinux.kallsyms1
KSYM5   .tmp_vmlinux.kallsyms1.5
AS       .tmp_vmlinux.kallsyms1.5
LD      .tmp_vmlinux.kallsyms2
KSYM5   .tmp_vmlinux.kallsyms2.5
AS       .tmp_vmlinux.kallsyms2.5
LD      vmlinux
SORTTAB vmlinux
SYSMAP  System.map
OBJCOPY arch/arm/boot/Image
MODPOST modules-only.symvers
GEN      Module.symvers
Kernel: arch/arm/boot/Image is ready
LDS      arch/arm/boot/compressed/vmlinux.lds
AS       arch/arm/boot/compressed/head.o
GZIP     arch/arm/boot/compressed/piggy_data
CC       arch/arm/boot/compressed/misc.o
CC       arch/arm/boot/compressed/decompress.o
CC       arch/arm/boot/compressed/string.o
SHIPPED  arch/arm/boot/compressed/hyp-stub.S
CC       arch/arm/boot/compressed/fdt_rw.o
CC       arch/arm/boot/compressed/fdt_ro.o
CC       arch/arm/boot/compressed/fdt_wip.o
CC       arch/arm/boot/compressed/fdt.o
CC       arch/arm/boot/compressed/fdt_check_mem_start.o
SHIPPED  arch/arm/boot/compressed/lib1funcs.S
SHIPPED  arch/arm/boot/compressed/ashldi3.S
SHIPPED  arch/arm/boot/compressed/bswapsdi2.S
AS       arch/arm/boot/compressed/hyp-stub.o
AS       arch/arm/boot/compressed/lib1funcs.o
AS       arch/arm/boot/compressed/ashldi3.o
AS       arch/arm/boot/compressed/bswapsdi2.o
AS       arch/arm/boot/compressed/piggy.o
LD       arch/arm/boot/compressed/vmlinux
OBJCOPY  arch/arm/boot/zImage
Kernel: arch/arm/boot/zImage is ready
nhatnguyen@nhatnguyen-Predator-PHN16-72:~/embedded_lab1/kernel/linux-5.15$ ls -lh arch/arm/boot/zImage
-rwxrwxr-x 1 nhatnguyen nhatnguyen 4,8M Thg 9 16 19:18 arch/arm/boot/zImage
nhatnguyen@nhatnguyen-Predator-PHN16-72:~/embedded_lab1/kernel/linux-5.15$ ls -lh arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rw-rw-r-- 1 nhatnguyen nhatnguyen 14K Thg 9 16 19:17 arch/arm/boot/dts/vexpress-v2p-ca9.dtb
nhatnguyen@nhatnguyen-Predator-PHN16-72:~/embedded_lab1/kernel/linux-5.15$ file arch/arm/boot/zImage
arch/arm/boot/zImage: Linux kernel ARM boot executable zImage (little-endian)
nhatnguyen@nhatnguyen-Predator-PHN16-72:~/embedded_lab1/kernel/linux-5.15$ arm-linux-gnueabihf-readelf -A arch/arm/boot/compressed/vmlinux | grep -i 'arch\|cpu'
Tag_CPU_name: "7-A"
Tag_CPU_arch: v7
Tag_CPU_arch_profile: Application
Tag_FP_arch: VFPv2
Tag_CPU_unaligned_access: v6
nhatnguyen@nhatnguyen-Predator-PHN16-72:~/embedded_lab1/kernel/linux-5.15$ cp arch/arm/boot/zImage ~/embedded_lab1/output/
nhatnguyen@nhatnguyen-Predator-PHN16-72:~/embedded_lab1/kernel/linux-5.15$ cp arch/arm/boot/dts/vexpress-v2p-ca9.dtb ~/embedded_lab1/output/
nhatnguyen@nhatnguyen-Predator-PHN16-72:~/embedded_lab1/kernel/linux-5.15$ ls -lh ~/embedded_lab1/output/
total 4,8M
-rw-rw-r-- 1 nhatnguyen nhatnguyen 14K Thg 9 16 19:23 vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 nhatnguyen nhatnguyen 4,8M Thg 9 16 19:23 zImage
nhatnguyen@nhatnguyen-Predator-PHN16-72:~/embedded_lab1/kernel/linux-5.15$
```

Phần B: Output của printenv, setenv bootargs, kết quả Kernel panic khi chưa có rootfs

```
In: serial
Out: serial
Err: serial
Net: eth0: ethernet@3,02000000
Hit any key to stop autoboot: 0
=> version
J-Boot 2022.04 (Sep 20 2026 - 12:11:27 +0700)

arm-linux-gnueabi-hf-gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
GNU ld (GNU Binutils for Ubuntu) 2.38
=> binfo
boot_params = 0x60002000
DRAM bank   = 0x00000000
-> start     = 0x60000000
-> size      = 0x20000000
DRAM bank   = 0x00000001
-> start     = 0x80000000
-> size      = 0x00000004
flashstart  = 0x40000000
flashsize   = 0x04000000
flashoffset = 0x00000000
baudrate    = 38400 bps
relocaddr   = 0x7ff6b000
reloc off   = 0x1f76b000
Build       = 32-bit
current eth = ethernet@3,02000000
ethaddr     = 52:54:00:12:34:56
IP addr     = <NULL>
fdt_blob    = 0x7ffe0120
new_fdt     = 0x00000000
fdt_size    = 0x00000000
lmb_dump_all:
memory.cnt  = 0x1
memory[0]   [0x60000000-0x80000003], 0x20000004 bytes flags: 0
reserved.cnt = 0x2
reserved[0] [0x4c000000-0x4c7ffffff], 0x00800000 bytes flags: 4
reserved[1] [0x7fb26c88-0x7fb26c8f], 0x00000008 bytes flags: 0
devicetree  = embed
arch_number = 0x000000e0
TLB addr    = 0x7fff0000
irq_sp      = 0x7fb2aec0
sp start    = 0x7fb2aeb0
Early malloc usage: 370 / 400
=> █
```

```
nhatnguyen@nhatnguyen-Predator-PHN16-72: ~/embedded_lab1/uboot/u-boot-2022.04
boot_syslinux_conf=extlinux/extlinux.conf
boot_targets=mmc1 mmc0 pxe dhcp
bootargs=console=tty0 console=ttyAMA0,38400n8
bootcmd=run distro_bootcmd; run bootflash
bootcmd_dhcp=devtype=dhcp; if dhcp ${scriptaddr} ${boot_script_dhcp}; then source ${scriptaddr}; fi;setenv e
fi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; then setenv efi_fdtfile ${soc}-${board}${boar
dver}.dtb; fi; setenv efi_old_vci ${bootp_vci};setenv efi_old_arch ${bootp_arch};setenv bootp_vci PXEClient:
Arch:00010:UNDI:003000;setenv bootp_arch 0xa;if dhcp ${kernel_addr_r}; then tftpboot ${fdt_addr_r} dtb/${efi
_fdtfile};if fdt addr ${fdt_addr_r}; then bootefi ${kernel_addr_r} ${fdt_addr_r}; else bootefi ${kernel_addr
_r} ${fdtcontroladdr};fi;fi;setenv bootp_vci ${efi_old_vci};setenv bootp_arch ${efi_old_arch};setenv efi_fdt
file;setenv efi_old_arch;setenv efi_old_vci;
bootcmd_mmc0=devnum=0; run mmc_boot
bootcmd_mmc1=devnum=1; run mmc_boot
bootcmd_pxe=dhcp; if pxe get; then pxe boot; fi
bootdelay=2
bootflash=run flashargs; cp ${ramdisk_addr} ${ramdisk_addr_r} ${maxramdisk}; bootm ${kernel_addr} ${ramdisk_
addr_r}
console=ttyAMA0,38400n8
cpu=armv7
distro_bootcmd=for target in ${boot_targets}; do run bootcmd_${target}; done
dram=1024M
efi_dtb_prefixes= /dtb/ /dtb/current/
ethaddr=52:54:00:12:34:56
fdt_addr_r=0x60000000
fdtcontroladdr=7ffe0120
fdtfile=vexpress-v2p-ca9.dtb
flashargs=setenv bootargs root=${root} console=${console} mem=${dram} mtdparts=${mtd} mmci.fmax=190000 devtm
pfs.mount=0 vmalloc=256M
kernel_addr_r=0x60100000
load_efi_dtb=load ${devtype} ${devnum}:${distro_bootpart} ${fdt_addr_r} ${prefix}${efi_fdtfile}
loadaddr=0x90000000
mmc_boot=if mmc dev ${devnum}; then devtype=mmc; run scan_dev_for_boot_part; fi
mtd=armflash:1M@0x8000000(uboot),7M@0x1000000(kernel),24M@0x2000000(initrd)
root=/dev/sda1 rw
scan_dev_for_boot=echo Scanning ${devtype} ${devnum}:${distro_bootpart}...; for prefix in ${boot_prefixes};
do run scan_dev_for_extlinux; run scan_dev_for_scripts; done;run scan_dev_for_efi;
scan_dev_for_boot_part=part list ${devtype} ${devnum} -bootable devplist; env exists devplist || setenv devp
list 1; for distro_bootpart in ${devplist}; do if fstype ${devtype} ${devnum}:${distro_bootpart} bootfstype;
then run scan_dev_for_boot; fi; done; setenv devplist
scan_dev_for_efi=setenv efi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; then setenv efi_fdtf
ile ${soc}-${board}${boardver}.dtb; fi; for prefix in ${efi_dtb_prefixes}; do if test -e ${devtype} ${devnum
}:${distro_bootpart} ${prefix}${efi_fdtfile}; then run load_efi_dtb; fi;done;run boot_efi_bootmgr;if test -e
${devtype} ${devnum}:${distro_bootpart} efi/boot/bootarm.efi; then echo Found EFI removable media binary ef
i/boot/bootarm.efi; run boot_efi_binary; echo EFI LOAD FAILED: continuing...; fi; setenv efi_fdtfile
scan_dev_for_extlinux=if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${boot_syslinux_conf}; the
n echo Found ${prefix}${boot_syslinux_conf}; run boot_extlinux; echo SCRIPT FAILED: continuing...; fi
scan_dev_for_scripts=for script in ${boot_scripts}; do if test -e ${devtype} ${devnum}:${distro_bootpart} ${
prefix}${script}; then echo Found U-Boot script ${prefix}${script}; run boot_a_script; echo SCRIPT FAILED: c
ontinuing...; fi; done
stderr=serial
stdin=serial
stdout=serial
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubivol || env set bootubivol bo
ot; if ubi part ${bootubipart} && ubismount ubi${devnum}:${bootubivol}; then devtype=ubi; run scan_dev_for_
boot; fi
vendor=armltd

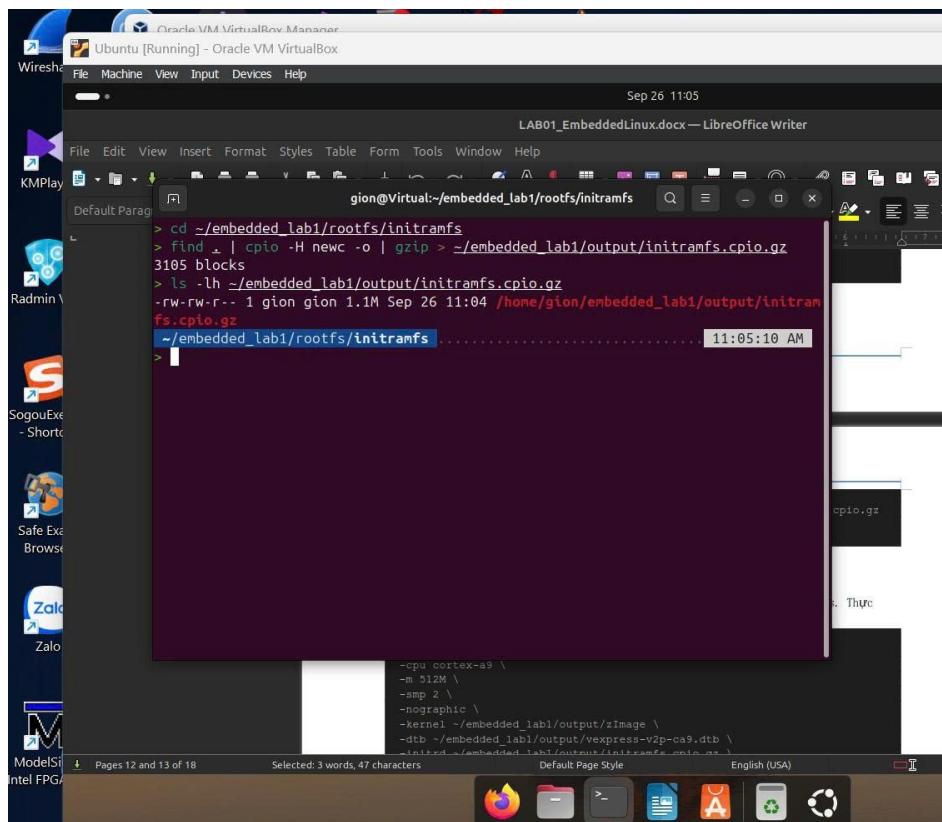
Environment size: 4238/262140 bytes
=>
```

```
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=> printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
=> setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'
=> saveenv
Saving Environment to Flash... Error: start and/or end address not on sector boundary
Error: start and/or end address not on sector boundary
Failed (1)
=>
```

```
nhatnguyen@nhatnguyen-Predator-PHN16-72: ~/embedded_lab1/u-boot/u-boot-2022.04

NET: Registered PF_PACKET protocol family
9pnet: Installing 9P2000 support
Registering SHP/SWPB emulation handler
aaci-pl041 10004000.aaci: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
aaci-pl041 10004000.aaci: FIFO 512 entries
mmci-pl18x 10005000.mmci: Got CD GPIO
mmci-pl18x 10005000.mmci: Got WP GPIO
mmci-pl18x 10005000.mmci: mmc0: PL181 nanf 41 rev0 at 0x10005000 irq 33,34 (pio)
10009000.uart: ttyAMA0 at MMIO 0x10009000 (irq = 37, base_baud = 0) is a PL011 rev1
printk: console [ttyAMA0] enabled
1000a000.uart: ttyAMA1 at MMIO 0x1000a000 (irq = 38, base_baud = 0) is a PL011 rev1
1000b000.uart: ttyAMA2 at MMIO 0x1000b000 (irq = 39, base_baud = 0) is a PL011 rev1
1000c000.uart: ttyAMA3 at MMIO 0x1000c000 (irq = 40, base_baud = 0) is a PL011 rev1
rtc-pl031 10017000.rtc: registered as rtc0
rtc-pl031 10017000.rtc: setting system clock to 2026-09-20T05:23:47 UTC (1789881827)
amba 10020000.clcd: Fixing up cyclic dependency with 0-0039
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: Initializing Versatile Express PL111
sii902x 0-0039: supply tvccv not found, using dummy regulator
sii902x 0-0039: supply cvcc12 not found, using dummy regulator
i2c i2c-0: Added multiplexed i2c bus 2
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express Init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: Initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: found bridge on endpoint 0
drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/notherboard-bus@40000000/notherboard-bus@40000000:iofpga7,00000000/10006000.kmt/serio0/input/input0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
  #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
input: InEXPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/notherboard-bus@40000000/notherboard-bus@40000000:iofpga7,00000000/10007000.kmt/serio1/input/input2
VFS: Cannot open root device "ram0" or unknown-block(0,0): error -6
Please append a correct "root=" boot option; here are the available partitions:
1f00          131072 mtdblock0
   (driver?)
1f01             32768 mtdblock1
   (driver?)
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1
Hardware name: ARM-Versatile Express
[<8010af20>] (unwind_backtrace) from [<8010af48>] (show_stack+0x10/0x14)
[<8010af48>] (show_stack) from [<8083b7f8>] (dump_stack_lvl+0x40/0x4c)
[<8083b7f8>] (dump_stack_lvl) from [<8083779c>] (panic+0xf8/0x2f4)
[<8083779c>] (panic) from [<80801705>] (mount_block_root+0x190/0x230)
[<80801705>] (mount_block_root) from [<80b01b80>] (prepare_namespace+0x150/0x18c)
[<80b01b80>] (prepare_namespace) from [<8083f8fc>] (kernel_init+0x10/0x124)
[<8083f8fc>] (kernel_init) from [<80100130>] (ret_from_fork+0x14/0x24)
Exception stack(0x810a7fb0 to 0x810a7ff8)
7fab: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fc0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fe0: 00000000 00000000 00000000 00000000 00000013 00000000 00000000 00000000
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ]---
```

Phần C: Output của ls initramfs/, lệnh boot QEMU thành công, kết quả các lệnh uname -a, mount, ps



```
uname -a
/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Wed Sep 16 19:17:29 +07 2026 armv7l GNU/Linux
/ # cat /proc/cpuinfo
processor       : 0
model name     : ARMv7 Processor rev 0 (v7l)
BogoMIPS      : 1705.57
Features       : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
CPU implementer : 0x41
CPU architecture: 7
CPU variant    : 0x0
CPU part       : 0xc09
CPU revision   : 0

processor       : 1
model name     : ARMv7 Processor rev 0 (v7l)
BogoMIPS      : 1731.78
Features       : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
CPU implementer : 0x41
CPU architecture: 7
CPU variant    : 0x0
CPU part       : 0xc09
CPU revision   : 0

Hardware       : ARM-Versatile Express
Revision      : 0000
Serial        : 0000000000000000
/ # cat /proc/cpuinfo | grep -E 'processor|model|Features'
processor       : 0
model name     : ARMv7 Processor rev 0 (v7l)
Features       : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
processor       : 1
model name     : ARMv7 Processor rev 0 (v7l)
Features       : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
/ # cat /proc/meminfo | head -5
MemTotal:      508288 kB
MemFree:       493168 kB
MemAvailable:  490396 kB
Buffers:       0 kB
Cached:        1524 kB
/ # ls /dev
console null    random zero
/ # mount
rootfs on / type rootfs (rw,size=244912k,nr_inodes=61228)
none on /proc type proc (rw,relatime)
none on /sys type sysfs (rw,relatime)
none on /tmp type tmpfs (rw,relatime)
```

```

/ # ps
PID    USER      TIME  COMMAND
  1  root        0:02  init
  2  root        0:00  [kthreadd]
  3  root        0:00  [rcu_gp]
  4  root        0:00  [rcu_par_gp]
  5  root        0:00  [kworker/0:0-eve]
  6  root        0:00  [kworker/0:0H]
  7  root        0:00  [kworker/u8:0-ev]
  8  root        0:00  [mm_percpu_wq]
  9  root        0:00  [ksoftirqd/0]
 10  root        0:00  [rcu_sched]
 11  root        0:00  [migration/0]
 12  root        0:00  [cpuhp/0]
 13  root        0:00  [cpuhp/1]
 14  root        0:00  [migration/1]
 15  root        0:00  [ksoftirqd/1]
 16  root        0:00  [kworker/1:0-rcu]
 17  root        0:00  [kworker/1:0H-ev]
 18  root        0:00  [cpuhp/2]
 19  root        0:00  [migration/2]
 20  root        0:00  [ksoftirqd/2]
 21  root        0:00  [kworker/2:0]
 22  root        0:00  [kworker/2:0H]
 23  root        0:00  [cpuhp/3]
 24  root        0:00  [kworker/0:1-rcu]
 25  root        0:00  [migration/3]
 26  root        0:00  [ksoftirqd/3]
 27  root        0:00  [kworker/3:0]
 28  root        0:00  [kworker/3:0H]
 29  root        0:00  [kdevtmpfs]
 30  root        0:00  [inet_frag_wq]
 31  root        0:00  [kworker/1:1-eve]
 32  root        0:00  [khungtaskd]
 33  root        0:00  [oom_reaper]
 34  root        0:00  [writeback]
 35  root        0:00  [kcompactd0]
 36  root        0:00  [kblockd]
 37  root        0:00  [ata_sff]
 38  root        0:00  [kworker/u8:1-ev]
 39  root        0:00  [rpciod]
 40  root        0:00  [kworker/u9:0]
 41  root        0:00  [xprtiod]
 42  root        0:00  [kworker/u8:2-ev]
 43  root        0:00  [kworker/1:1H]
 44  root        0:00  [kswapd0]
 45  root        0:00  [nfsiod]
 46  root        0:00  [kworker/1:2-eve]
 50  root        0:00  [irq/33-mmci-pl1]
 51  root        0:00  [kworker/1:3-rcu]
 52  root        0:00  [card0-crtc0]
 53  root        0:00  [kworker/0:2-eve]
 59  root        0:00  -/bin/sh
 68  root        0:00  ps
/ # cat /sys/devices/system/cpu/possible
0-3

```

Mục 4: PHÂN TÍCH VÀ TRẢ LỜI CÂU HỎI

Câu 1. So sánh hai toolchain arm-linux-gnueabi và arm-linux-gnueabihf: chúng khác nhau ở điểm gì về ABI và FPU? Tại sao với Cortex-A9 (ARMv7-A) ta phải dùng gnueabihf thay vì gnueabi? Điều gì xảy ra nếu build kernel bằng gnueabi rồi chạy trên Cortex-A9?

arm-linux-gnueabi (Soft-Float / SoftFP): Sử dụng giao thức EABI cũ. Các tính toán floating-point được thực hiện bằng phần mềm mô phỏng (soft-float) hoặc truyền tham số qua các thanh ghi số nguyên. Hiệu năng số thực chậm hơn do tốn chi phí chuyển đổi dữ liệu nhưng độ tương thích lại rất cao, chạy được trên hầu hết chip ARM cũ.

arm-linux-gnueabihf (Hard-Float): Sử dụng giao thức EABI hard-float. Hậu tố hf trong gnueabihf có nghĩa là Hard Float. Tham số số thực được truyền trực tiếp qua các thanh ghi phần cứng VFP/NEON (d0-d7, s0-s15), giúp tăng tốc độ tính toán lên gấp nhiều lần vì được xử lý trực tiếp trên phần cứng.

Tại sao Cortex-A9 phải dùng gnueabihf: Cortex-A9 (kiến trúc ARMv7-A) được trang bị bộ xử lý số thực phần cứng (VFPv3 / NEON FPU) có khả năng thực hiện các phép toán floating-point bằng phần cứng. Việc dùng gnueabihf tận dụng trực tiếp phần cứng FPU, tối ưu hiệu năng và tránh lãng phí chu kỳ CPU cho việc mô phỏng phần mềm.

Nếu build Kernel bằng gnueabi chạy trên Cortex-A9: Kernel và ứng dụng vẫn có thể chạy được do CPU hỗ trợ tương thích ngược, nhưng hiệu năng các tác vụ số thực sẽ giảm rất mạnh. Điều nghiêm trọng xảy ra userspace và các shared libraries sử dụng hai ABI khác nhau. Ví dụ, nếu BusyBox được build bằng gnueabihf nhưng root filesystem lại chứa thư viện soft-float, chương trình có thể không chạy do ABI không tương thích và lỗi Crash (Incompatible ABI).

Câu 2. Giải thích tại sao memory map của vexpress-a9 bắt đầu từ 0x60000000 thay vì 0x00000000 như Versatile PB. Điều này ảnh hưởng như thế nào đến cách U-Boot thiết lập bootcmd (địa chỉ load kernel, DTB, initramfs)? Nếu đặt sai địa chỉ load, điều gì xảy ra?

Tại sao bắt đầu từ 0x60000000: Versatile PB được thiết kế dựa trên các vi xử lý ARM đời cũ. Bản đồ bộ nhớ của hệ thống này đặt bộ nhớ RAM

tĩnh/động (DRAM) ở vùng thấp nhất của không gian địa chỉ, hệ thống ARM Versatile Express (vexpress-a9) thiết kế không gian địa chỉ NOR Flash/SRAM ở các vùng địa chỉ thấp và dành dải địa chỉ vật lý từ 0x60000000 trở đi cho SDRAM (DDR RAM).

Ảnh hưởng đến bootcmd trong U-Boot: Khi nạp zImage, DTB và initramfs vào RAM, U-Boot bắt buộc phải nạp vào các địa chỉ hợp lệ nằm trong vùng RAM vật lý từ 0x60000000 đến 0x80000000 (cho 512MB RAM):

Kernel Load Address: 0x60100000

DTB Load Address: 0x60000000

Initramfs Load Address: 0x64000000

Nếu đặt sai địa chỉ load:

Nạp vào dải < 0x60000000: Gây lỗi Data Abort / Bus Error do truy cập vào vùng nhớ không tồn tại hoặc vùng I/O bị ghi đè.

Nạp các thành phần đè lên nhau (ví dụ địa chỉ initramfs quá gần zImage): Khi zImage tự giải nén (uncompress), nó sẽ ghi đè lên initramfs hoặc DTB, dẫn đến Kernel Panic hoặc treo hệ thống.

Câu 3. Trong boot log của ARMv7, bạn thấy dòng 'CPU: ARMv7 Processor [410fc090]'. Giải thích ý nghĩa của MIDR (Main ID Register) 0x410fc090: implementer (0x41), variant, architecture, part number (0xc09), revision là gì? Tra cứu và giải thích part number 0xc09 tương ứng với CPU nào.

Mã Hex 0x410fc090 (32-bit) đại diện cho thông tin trong thanh ghi Main ID Register (MIDR) của kiến trúc ARM:

Implementer (0x41 - Bits): Mã của hãng thiết kế / sản xuất (ASCII 'A' = ARM Ltd).

Variant (0x0 - Bits): Phiên bản biến thể của lõi CPU (Variant 0).

Architecture (0xF - Bits): Mã kiến trúc CPU (0xF chỉ định phiên bản mô tả qua tính năng / ARMv7).

Part Number (0xC09 - Bits [15:4]): Mã định danh dòng chip. Tra cứu mã 0xC09 chính là ARM Cortex-A9.

Revision (0x0 - Bits [3:0]): Revision r0p0 của lõi silicon.

Câu 4. Tại sao cần build BusyBox với static linking trong hệ thống initramfs tối giản? Nếu build dynamic linking cho ARMv7, cần copy những shared library nào từ toolchain vào rootfs? Liệt kê ít nhất 3 file .so cần thiết và cho biết chúng nằm ở đâu trong toolchain.

Tại sao cần Static Linking cho Initramfs tối giản: Khi hệ thống vừa khởi động, initramfs hoạt động như một RootFS tạm thời độc lập. Nếu biên dịch động (dynamic), Linux ELF loader (ld-linux.so) không tìm thấy các thư viện liên kết động (.so), hệ thống sẽ lập tức báo lỗi Kernel panic - not syncing: Attempted to kill init! hoặc Failed to execute /init. Static linking đóng gói toàn bộ hàm hệ thống vào file thực thi duy nhất, giúp RootFS không cần bất kỳ thư viện bên ngoài nào.

Nếu dùng Dynamic Linking, 3 file .so bắt buộc phải copy từ Toolchain vào /lib:

ld-linux-armhf.so.3 (Dynamic Linker / Loader) - Vị trí:
sysroot/lib/ld-linux-armhf.so.3

libc.so.6 (C Standard Library) - Vị trí: sysroot/lib/libc.so.6

libm.so.6 (Math Library) - Vị trí: sysroot/lib/libm.so.6

Câu 5. Khi bật tùy chọn SMP (Symmetric Multi-Processing) trong kernel config cho vexpress-a9 với -smp 2, kernel cần khởi tạo thêm những gì so với uniprocessor? Quan sát sự khác biệt trong boot log giữa -smp 1 và -smp 2. Nêu ít nhất 2 dòng log thay đổi và giải thích ý nghĩa.

Các công việc Kernel cần khởi tạo bổ sung khi bật SMP:

Khởi tạo GIC (Generic Interrupt Controller) hỗ trợ IPI (Inter-Processor Interrupts) để các lõi gửi ngắt giao tiếp với nhau.

Bật SCU (Snoop Control Unit) để duy trì tính nhất quán của bộ nhớ đệm (Cache Coherency) giữa CPU0 và CPU1.

CPU0 gửi tín hiệu khởi động (secondary_boot / PSCI call) đưa CPU1 thoát khỏi trạng thái WFE (Wait For Event) và thiết lập con trỏ stack cho CPU1.

2 Dòng log thay đổi rõ rệt trong Boot Log:

Dòng log 1 (Phát hiện và Khởi động Lõi phụ):

Với -smp 1: SMP: Total of 1 processors activated (....
BogoMIPS)

Với -smp 2: SMP: Bringing up secondary CPUs ... và CPU1:
Booted secondary processor [410fc090]

Dòng log 2 (Khởi tạo Cores):

Với -smp 2: Brought up 1 node, 2 CPUs (Kernel nhận diện hoàn
chỉnh 2 CPU core).

Mục 5: NHẬN XÉT & KHÓ KHĂN GẶP PHẢI

1. Lỗi Unexpected hard-float flag set in e_flags khi build Kernel

Khi biên dịch Linux Kernel với các tùy chọn:

```
export KCFLAGS="-march=armv7-a -mfpu=vfpv3 -mfloat-abi=hard"
```

quá trình build xuất hiện lỗi:

```
vdsomunge: Unexpected hard-float flag set in e_flags
```

Nguyên nhân là cấu hình hard-float được truyền trực tiếp vào quá trình build Kernel khiến một số thành phần của Kernel/VDSO không tương thích với cờ floating-point này.

Để khắc phục, thay đổi KCFLAGS thành:

```
export KCFLAGS="-march=armv7-a"
```

sau đó thực hiện build lại:

```
make -j$(nproc) zImage dtbs modules
```

```
make modules_prepare
```

Kết quả Kernel được build thành công và tạo ra:

```
arch/arm/boot/zImage
```

```
arch/arm/boot/dts/vexpress-v2p-ca9.dtb
```

2. Lỗi liên quan đến địa chỉ bộ nhớ khi cấu hình U-Boot

Khi cấu hình U-Boot cho vexpress-a9, cần sử dụng đúng địa chỉ bộ nhớ của nền tảng QEMU. Trong Lab, các địa chỉ được sử dụng để boot là:

```
0x60100000 → zImage
```

```
0x64000000 → initramfs
```

```
0x60000000 → DTB
```

Ban đầu U-Boot tự động thực hiện `distro_bootcmd` và tìm cách boot qua các nguồn như network/TFTP, dẫn đến các thông báo lỗi liên quan đến TFTP và image.

Để khắc phục, em thiết lập lại các biến môi trường U-Boot:

```
setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw  
init=/sbin/init mem=512M'
```

```
setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'
```

Sau đó hệ thống có thể sử dụng đúng Kernel, initramfs và Device Tree để khởi động.

Cuối cùng, em kiểm tra trực tiếp bằng QEMU và hệ thống khởi động thành công đến BusyBox shell với prompt: `/ #`